

Unidade 2

Explorando Recursos do Python

Aula 1

Estruturas de Dados em Python - Parte I

Estruturas de dados em Python – parte I

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Em Python, a premissa fundamental é a de que tudo gira em torno de objetos. De acordo com a Python Software Foundation (PSF), todos os dados em um programa Python são representados por objetos ou pela relação entre objetos. Conheceremos, nesta aula, três estruturas de dados: sequência, lista e tuplas.

Sequências são estruturas de dados que nos permitem armazenar coleções ordenadas de informações. As listas consistem em uma forma fundamental de objetos do tipo sequência e são mutáveis, o que significa que nesse caso podemos adicionar, remover e alterar elementos. Já as

tuplas são praticamente semelhantes às listas, mas com uma diferença crucial: elas são imutáveis. Isso significa que, uma vez criadas, as tuplas não podem ser alteradas.

Para contextualizar sua aprendizagem, imagine a seguinte situação: você está gerenciando a lista de convidados de uma festa e a lista de pessoas que confirmaram a presença no evento. Você deseja identificar as pessoas que ainda não confirmaram presença, a fim de convidá-las novamente.

Vamos Começar!

Objetos do tipo sequência

Os objetos do tipo sequência são como coleções versáteis que podem armazenar vários valores. Eles servem para organizar dados em uma ordem específica e são indexados por números inteiros não negativos. O primeiro elemento da sequência é acessado pelo índice 0, o segundo, pelo índice 1, e assim por diante, até o último elemento, que está na posição $n - 1$, onde n representa a capacidade de armazenamento da sequência. O grupo de estruturas de dados que se encaixam nessa categoria compartilha algumas operações comuns. Observe o Quadro 1, a seguir.

Operação	Resultado
$x \text{ in } s$	True caso um item de s seja igual a x , senão false.
$s + t$	Concatenação de s e t .
$n * s$	Adiciona s a si mesmo n vezes.
$s[i]$	Acessa o valor guardado na posição i da sequência.
$s[i:j]$	Acessa os valores da posição i até j .
$s[i:j:k]$	Acessa os valores da posição i até j , com passo k .
$\text{len}(s)$	Comprimento de s .
$\text{min}(s)$	Menor valor de s .
$\text{max}(s)$	Maior valor de s .
$s.\text{count}(x)$	Número total de ocorrência de x em s .

Quadro 1 | Operações em comum dos objetos do tipo sequência. Fonte: adaptado de PSF.

Um texto, representado por objetos da classe str (strings), é uma forma de sequência. Essas strings oferecem uma variedade de operações, como as descritas no Quadro 1, apresentado anteriormente. No entanto, é importante notar que as strings são objetos imutáveis, o que significa que não é possível modificar seu conteúdo atribuindo um novo valor a uma posição específica. Vamos, agora, experimentar algumas dessas operações. Confira o código a seguir.

```
texto = "Explorando a diversidade de linguagens de programação com Python."

print(f"Tamanho do texto = {len(texto)}")
print(f"Python in texto = {'Python' in texto}")
print(f"Quantidade de e no texto = {texto.count('e')}")
print(f"As 5 primeiras letras são: {texto[:5]}")
#resultado
Tamanho do texto = 62
Python in texto = False
Quantidade de e no texto = 6
As 5 primeiras letras são: Explo
```

Na primeira demonstração, exploramos várias operações que podem ser aplicadas a sequências. A função **len()** revela o tamanho da sequência, enquanto o operador "in" permite verificar a presença de um valor na sequência. Com o operador **count**, é possível determinar quantas vezes um valor específico aparece na sequência. Além disso, usando a notação de colchetes, podem-se extrair partes específicas da sequência, como demonstrado na linha 6, onde solicitamos a exibição dos elementos da posição 0 até 5, excluindo o valor na posição 6.

A classe str (strings) vai além das operações listadas no Quadro 1, sugerindo uma série de outros métodos úteis. O site da Python Software Foundation (PSF) contém uma lista completa dessas funções para objetos str.

Listas

As listas são estruturas de dados em Python conhecidas por sua mutabilidade, o que significa que você pode adicionar ou remover elementos conforme necessário. São estruturas indexadas, ou seja, cada elemento tem uma posição, começando em 0.

Considere o código a seguir, no qual criamos uma lista chamada "cores" e, em seguida, usamos uma estrutura de repetição para imprimir cada elemento junto com seu índice. Observe a função **index**, que retorna a posição de um valor na lista.

```
cores = ['vermelho', 'azul', 'verde', 'amarelo', 'roxo']
for cor in cores:
    print(f"Posição = {cores.index(cor)}, cor = {cor}")
```

```
#resultado
Posição = 0, cor = vermelho
Posição = 1, cor = azul
Posição = 2, cor = verde
Posição = 3, cor = amarelo
Posição = 4, cor = roxo
```

As list comprehensions, ou listcomps, são uma abordagem pythônica para criar listas com base em objetos iteráveis. Essa técnica é especialmente útil quando você deseja transformar ou filtrar as informações de uma sequência existente para construir uma nova sequência com as informações desejadas. Para ilustrar essa técnica, vamos considerar um exemplo no qual temos uma lista de palavras e queremos convertê-las em letras minúsculas. Acompanhe o código a seguir:

```
linguagens = ["Python", "Java", "JavaScript", "C", "C#", "C++", "Swift", "Go", "Kotlin"]
print("Antes da listcomp = ", linguagens)
linguagens = [item.lower() for item in linguagens]
print("\nDepois da listcomp = ", linguagens)
#resultado
Antes da listcomp = ['Python', 'Java', 'JavaScript', 'C', 'C#', 'C++', 'Swift', 'Go', 'Kotlin']
Depois da listcomp = ['python', 'java', 'javascript', 'c', 'c#', 'c++', 'swift', 'go', 'kotlin']
```

No exemplo apresentado anteriormente, criamos a lista "linguagens", que contém várias linguagens de programação. Em seguida, aplicamos uma list comprehension. No interior dos colchetes, utilizamos a variável "item" para representar cada elemento da lista original. Com a expressão "item.lower()", transformamos cada elemento em minúsculas e substituímos os valores originais na mesma variável "linguagens". Por fim, imprimimos a lista antes e depois da aplicação da list comprehension.

Agora, vamos explorar as funções **map()** e **filter()** em Python, que são usadas para manipular listas e aplicar transformações ou filtragens a elementos iteráveis. Primeiro, vou apresentar exemplos diferentes para cada função.

Suponha que você tenha uma lista de preços em dólares e deseje convertê-los para reais usando uma taxa de câmbio fixa:

```
precos_em_dolares = [100, 50, 75, 120]
taxa_de_cambio = 5.25
precos_em_reais = list(map(lambda x: x * taxa_de_cambio, precos_em_dolares))
print(precos_em_reais)
#Resultado: [525.0, 262.5, 393.75, 630.0]
```

Nesse caso, usamos a função **map()** para aplicar uma função lambda que multiplica cada preço em dólares pela taxa de câmbio. Depois, convertemos o resultado em uma lista. O resultado será uma lista com os preços em reais.

Agora, imagine que você tenha uma lista de números e queira filtrar apenas os números pares:

```
numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
numeros_pares = list(filter(lambda x: x % 2 == 0, numeros))
print(numeros_pares)
#Resultado: [2, 4, 6, 8, 10]
```

Nesse exemplo, usamos a função **filter()** com uma função lambda que verifica se um número é par (resto da divisão por 2 igual a 0) e, em seguida, convertemos o resultado em uma lista. O resultado será uma lista contendo apenas os números pares.

Siga em Frente...

Tuplas

As tuplas são estruturas de dados pertencentes ao grupo de objetos do tipo sequência em Python. A principal distinção entre listas e tuplas é o fato de que as listas são mutáveis, permitindo a atribuição de valores a posições específicas, enquanto as tuplas são objetos imutáveis.

Você pode criar tuplas em Python de três maneiras:

1. Usando um par de parênteses para denotar uma tupla vazia: **tupla1 = ()**.
2. Usando um par de parênteses e elementos separados por vírgulas: **tupla2 = ('a', 'b', 'c')**.
3. Usando o construtor de tipo **tuple()**.

Confira, a seguir, um exemplo no qual criamos uma tupla chamada "vogais" e, posteriormente, usamos uma estrutura de repetição para imprimir cada elemento da tupla, juntamente com sua posição:

```
vogais = ('a', 'e', 'i', 'o', 'u')
print(f"Tipo do objeto vogais = {type(vogais)}")
for p, x in enumerate(vogais):
    print(f"Posição = {p}, valor = {x}")
```

```
#Resultado:  
Tipo do objeto vogais = <class 'tuple'>  
Posição = 0, valor = a  
Posição = 1, valor = e  
Posição = 2, valor = i  
Posição = 3, valor = o  
Posição = 4, valor = u
```

Nesse exemplo, utilizamos a função **enumerate()** para obter tanto a posição quanto o valor de cada elemento na tupla. É importante observar que as tuplas são imutáveis, o que significa que, uma vez criadas, não é possível alterar seu conteúdo. Isso as torna úteis em situações nas quais a ordem dos elementos deve permanecer inalterada. Além disso, as tuplas têm um papel fundamental em várias operações em Python, como no desempacotamento de valores e no retorno múltiplo de funções.

Vamos Exercitar?

Vamos, agora, colocar em prática o que aprendemos pensando no problema apresentado no início desta aula. Cada venda é registrada como uma tupla com os seguintes elementos: data da venda, nome do produto, quantidade vendida e preço unitário. Essas tuplas são armazenadas em uma lista chamada **registros_de_vendas**. Além disso, você recebeu uma lista de produtos que precisam ser reabastecidos no estoque, chamada **produtos_a_reabastecer**. Também é preciso acompanhar o total de vendas de cada produto. Para fazer isso, você deve criar um dicionário chamado **total_de_vendas_por_produto**, no qual as chaves são os nomes dos produtos, e os valores são os totais de vendas para cada um. Vamos ao código!

```
# Tupla de convidados  
convidados = ("Alice", "Bob", "Carol", "David", "Eve")  
# Lista de confirmações  
confirmados = ["Bob", "David"]  
# Identificar quem ainda não confirmou  
nao_confirmados = [convidado for convidado in convidados if convidado not in confirmados]  
# Exibir os convidados que ainda não confirmaram  
print("Convidados que ainda não confirmaram:")  
for pessoa in nao_confirmados:  
    print(pessoa)  
# Enviar lembretes aos não confirmados  
print("\nEnviando lembretes para os convidados que ainda não confirmaram.")  
#Resultado:  
Convidados que ainda não confirmaram:  
Alice  
Carol
```

Eve

Enviando lembretes para os convidados que ainda não confirmaram.

Nesse código, temos uma tupla de convidados e uma lista de pessoas que confirmaram. Usamos tupla, lista e uma compreensão de lista (list comprehension) para identificar as pessoas que ainda não confirmaram. Em seguida, exibimos os nomes dessas pessoas e, opcionalmente, podemos enviar lembretes a elas. Essa situação utiliza objetos do tipo sequência (listas e tuplas) para resolver um problema prático.

Espero que você tenha gostado da solução. Lembre-se: a prática é importante! Mude alguma parte desse código e diversifique seu conhecimento!

Saiba mais

1. Para exercitar os conhecimentos aprendidos nesta aula, faça a leitura do livro [*Python 3: conceitos e aplicações: uma abordagem didática*](#), cujo link de acesso está disponível a seguir.

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book.

2. Como mencionado anteriormente, uma leitura interessante para quem está começando a programar em Python é a do livro [*Começando a programar em Python para leigos*](#).

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book.

3. Também encorajo você a navegar pelo site [Python](#), que contém documentações e definições sobre as ferramentas de Python.

Referências

5. ESTRUTURAS de dados. **Python 3.12.2 Documentation**, 9 dez. 2019. Disponível em: <https://docs.python.org/pt-br/3/tutorial/datastructures.html>. Acesso em: 21 out. 2023.

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/books/9788536530253>. Acesso em: 21 out. 2023.

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos**: lógica para desenvolvimento de programação de computadores. 29. ed. São Paulo: Érica, 2019.

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555202298>. Acesso em: 21 out. 2023.

PSF landing. **Python Software Foundation**, 12 dez. 2023. Disponível em: <https://www.python.org/psf-landing/>. Acesso em: 21 fev. 2024.

Aula 2

Estruturas de Dados em Python - Parte II

Estruturas de dados em Python – parte II

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Dando continuidade à nossa aprendizagem, nesta aula vamos aprofundar nossos conhecimentos sobre estruturas de objetos em Python.

O primeiro objeto a ser estudado é o do tipo `set` em Python. Um conjunto, ou `set`, é uma estrutura de dados que representa uma coleção de elementos únicos, sem repetição. Nesse sentido, descobriremos como criar, modificar e realizar operações com conjuntos.

O segundo objeto analisado será o do tipo `mapping`, com foco voltado ao dicionário (`dict`) em Python. Dicionários são estruturas que associam chaves a valores, permitindo o armazenamento e a recuperação eficiente de informações. Sabemos como criar dicionários, adicionar itens e efetuar operações de busca.

O terceiro objeto é do tipo `array NumPy`. O `NumPy` é uma biblioteca essencial para a computação científica em Python, fornecendo recursos avançados para manipular arrays multidimensionais. Entenderemos como criar, realizar operações e acessar elementos em arrays `NumPy`.

Para estimular a compreensão desses conteúdos, suponha que você esteja gerenciando um evento científico para o qual participantes de diferentes regiões do mundo se inscreveram. Cada participante concedeu informações sobre sua localização, afiliação a instituições de pesquisa e áreas de interesse. O objetivo é desenvolver análises sobre a distribuição geográfica dos participantes, suas afiliações e as áreas de interesse predominantes. Como podemos utilizar os conhecimentos desta aula para resolver esse caso?

Vamos Começar!

Objetos do tipo `set`

A palavra “`set`”, em Python, nos conduz diretamente à essência de uma estrutura de dados que se assemelha a conjuntos matemáticos. Os objetos do tipo “`set`” habilitam operações de conjuntos, como união, interseção, diferença e muitas outras. Essa estrutura é especialmente útil para realizar testes de associação e eliminar valores duplicados em uma sequência (PSF, 2020).

Além das operações familiares que já conhecemos para sequências, como `len(s)`, `x in s` e `x not in s`, os conjuntos oferecem funcionalidades adicionais.

Podemos agregar um novo elemento a um conjunto usando `add(valor)` e remover elementos com `remove(valor)`. Para explorar a lista completa de funções disponíveis, acesse: [python](https://docs.python.org/3/library/stdtypes.html#set-objects).

Em Python, existem duas formas principais de criar objetos do tipo “`set`”:

Usando um par de chaves e elementos separados por vírgulas, por exemplo: `set1 = {'a', 'b', 'c'}`.

Usando o construtor de tipo `set(iterable)` com um objeto iterável, como uma lista, uma tupla ou mesmo uma sequência de caracteres (`string`).

Confira, a seguir, um exemplo de criação de conjuntos:

```
# Criando um conjunto vazio
meu_conjunto = set()
# Adicionando elementos ao conjunto
meu_conjunto.add(10)
meu_conjunto.add(20)
meu_conjunto.add(30)
# Imprimindo o conjunto
print("Conjunto após adicionar elementos:", meu_conjunto)
# Verificando se um elemento está no conjunto
elemento = 20
if elemento in meu_conjunto:
    print(f"{elemento} está no conjunto.")
else:
    print(f"{elemento} não está no conjunto.")
# Removendo um elemento do conjunto
meu_conjunto.remove(20)
# Imprimindo o conjunto atualizado
print("Conjunto após remover o elemento 20:", meu_conjunto)
```

Nesse código, criamos um conjunto vazio, chamado **meu_conjunto**, e adicionamos elementos a ele usando o método **add()**. Em seguida, verificamos se um elemento específico está no conjunto utilizando a instrução **in**. Por fim, removemos um elemento com o método **remove()** e imprimimos o conjunto atualizado. Conjuntos são úteis para armazenar valores únicos e efetuar operações de pertencimento.

Objetos do tipo mapping

As estruturas de dados que estabelecem uma relação entre chaves e valores são conhecidas como objetos do tipo mapping. Em Python, o principal objeto que atende a essa propriedade é o dicionário, representado pelo tipo **dict**. Dicionários são mutáveis, o que significa que podemos modificar o valor associado a uma chave existente ou adicionar novas chaves.

Podemos criar dicionários em Python das seguintes maneiras:

- Usando um par de chaves para denotar um dicionário vazio: **dicionario1 = {}**.
- Usando pares de elementos na forma "chave: valor" separados por vírgulas: **dicionario2 = {'um': 1, 'dois': 2, 'três': 3}**.
- Usando o construtor de tipo **dict()**.

Observe, a seguir, alguns exemplos desses diferentes modos de criar um dicionário:

```
# Exemplo 1 - Criação de um dicionário vazio, seguido de atribuição de chaves e valores
dici_1 = {}
dici_1['nome'] = "Maria"
dici_1['idade'] = 25
# Exemplo 2 - Criação de um dicionário com pares chave: valor
dici_2 = {'nome': 'Maria', 'idade': 25}
# Exemplo 3 - Criação de um dicionário com uma lista de tuplas representando pares chave:
valor
dici_3 = dict([('nome', "Maria"), ('idade', 25)])
# Exemplo 4 - Criação de um dicionário usando a função built-in zip() e duas listas, uma para
as chaves e outra para os valores
dici_4 = dict(zip(['nome', 'idade'], ["Maria", 25]))
# Teste se todas as construções resultam em objetos iguais
print(dici_1 == dici_2 == dici_3 == dici_4) # Deve imprimir True
print(dici_1)
#Resultado:
True
{'nome': 'Maria', 'idade': 25}
```

Mostramos quatro maneiras distintas de criar dicionários e atribuir valores a eles. Para acessar um valor em um dicionário, use a notação **nome_dicionario[chave]**. Já para atribuir um novo valor, utilize **nome_dicionario[chave] = novo_valor**. Dicionários são úteis para armazenar informações associadas por chaves exclusivas.

Siga em Frente...

Objetos do tipo array NumPy

As estruturas de dados em Python abrangem uma ampla variedade de objetos e bibliotecas, cada qual projetado para funções específicas. Um recurso poderoso que se destaca nesse contexto é a biblioteca **NumPy**, desenvolvida para suportar a computação científica com Python. **NumPy** oferece uma vasta gama de funcionalidades, incluindo arrays multidimensionais e funções sofisticadas. Além disso, disponibiliza ferramentas para integração com código em C/C++ e Fortran, bem como recursos essenciais de álgebra linear, transformada de Fourier e geração de números aleatórios.

Para começar a utilizar o **NumPy**, é necessário instalá-lo no ambiente Python. Você pode fazer isso facilmente com o comando **pip install numpy**. Em plataformas como o Anaconda ou Google Colab, o NumPy já está incluído. Depois de instalado, você deve importar a biblioteca em seu

projeto usando o comando **import numpy** sempre que quiser aproveitar seus recursos vantajosos.

A biblioteca **NumPy** é particularmente valiosa para cientistas de dados e desenvolvedores de soluções de inteligência artificial, pois permite lidar de maneira eficiente com matrizes de dados complexas e realizar operações avançadas. Se você estiver interessado em aprender mais sobre o NumPy, é interessante verificar a documentação completa dessa biblioteca em [numpy](https://numpy.org/doc/stable/).

Confira o código a seguir:

```
# Importe a biblioteca NumPy
import numpy as np
# Crie um array NumPy de números inteiros
my_array = np.array([1, 2, 3, 4, 5])
# Imprima o array
print("Array original:")
print(my_array)
# Realize operações matemáticas com o array
squared_array = my_array ** 2 # Eleva cada elemento ao quadrado
sum_of_elements = np.sum(my_array) # Calcula a soma de todos os elementos
# Imprima os resultados
print("\nArray ao quadrado:")
print(squared_array)
print("\nSoma dos elementos:")
print(sum_of_elements)
# Acesse elementos do array
element_at_index_2 = my_array[2] # Acessa o elemento no índice 2
print("\nElemento no índice 2:", element_at_index_2)
#Resultado
Array original:
[1 2 3 4 5]
Array ao quadrado:
[1 4 9 16 25]
Soma dos elementos:
15
Elemento no índice 2: 3
```

Nesse código, importamos o NumPy como **np**, criamos um array NumPy chamado **my_array**, realizamos operações matemáticas nele e acessamos elementos por índice. O NumPy oferece uma maneira eficiente de trabalhar com matrizes e executar operações em massa.

Vamos Exercitar?

Agora, vamos colocar em prática o que aprendemos nesta aula para resolver nosso problema inicial. Como podemos usar conjuntos (sets) para identificar as diferentes regiões dos participantes do evento científico, dicionários para categorizar suas afiliações e arrays NumPy para analisar as áreas de interesse?

```
# Importe as bibliotecas necessárias
import numpy as np
# Dados dos participantes
participantes = [
    {
        "nome": "Alice",
        "localizacao": "EUA",
        "afiliacao": "Universidade A",
        "interesses": ["Física", "Astronomia"]
    },
    {
        "nome": "Bob",
        "localizacao": "Brasil",
        "afiliacao": "Instituto B",
        "interesses": ["Biologia", "Astronomia"]
    },
    {
        "nome": "Charlie",
        "localizacao": "Índia",
        "afiliacao": "Instituto C",
        "interesses": ["Química", "Engenharia"]
    }
]
# Adicione mais participantes conforme necessário

# Usando sets para identificar diferentes regiões dos participantes
regioes = set(participante["localizacao"] for participante in participantes)
# Usando um dicionário para categorizar afiliações
afiliacoes = {}
for participante in participantes:
    afiliacao = participante["afiliacao"]
    if afiliacao not in afiliacoes:
        afiliacoes[afiliacao] = []
    afiliacoes[afiliacao].append(participante["nome"])
# Usando NumPy para analisar áreas de interesse
areas_de_interesse = np.array([interesse for participante in participantes for interesse in
participante["interesses"]])
```

```
interesses_unicos, contagem = np.unique(areas_de_interesse, return_counts=True)
area_mais_popular = interesses_unicos[np.argmax(contagem)]
# Resultados
print("Regiões dos participantes:", regioes)
print("Afiliações dos participantes:")
for afiliacao, nomes in afiliacoes.items():
    print(f"{afiliacao}: {' '.join(nomes)}")
print("Área de interesse mais popular:", area_mais_popular)
#Resultado:
Regiões dos participantes: {'Índia', 'EUA', 'Brasil'}
Afiliações dos participantes:
Universidade A: Alice
Instituto B: Bob
Instituto C: Charlie
Área de interesse mais popular: Astronomia
```

Esse código usa conjuntos para identificar as diferentes regiões dos participantes, um dicionário para categorizar suas afiliações e o NumPy para avaliar as áreas de interesse. Os resultados são exibidos no final do código.

Gostou dessa solução? Espero que sim! Já estamos avançando bastante em nossa trajetória de estudos, desta vez utilizando bibliotecas do Python. Faça mudanças no código e pratique!

Saiba mais

1. Para entender mais detalhes sobre o uso do NumPy, sugiro a leitura do artigo [Os principais setores de emprego na Mesorregião do Sul/Sudoeste de Minas: uma análise multivariada](#), que exibe uma análise empírica quantitativa cujo objetivo foi verificar como se comportam os índices de empregabilidade utilizando Python.

PAPANDRÉA, P. J.; PEREIRA, A. de S.; PAIVA, A. P. de. Os principais setores de emprego na Mesorregião do Sul/Sudoeste de Minas: uma análise multivariada. **Produção Online**, Florianópolis, SC, v. 22, n. 4, p. 3528-3554, 2022.

2. Para exercitar os conhecimentos aprendidos nesta aula, faça a leitura do livro [Python 3: conceitos e aplicações: uma abordagem didática](#).

BANIN, S. L. **Python 3: conceitos e aplicações: uma abordagem didática**. São Paulo: Érica, 2018. E-book.

3. Outra dica para estudo e aprofundamento sobre esse tema é o livro [Use a cabeça! Python](#).

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book.

Referências

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos**: lógica para desenvolvimento de programação de computadores. 29. ed. São Paulo: Érica, 2019.

NUMPY. **Página inicial**, [s. d.]. Disponível em: <https://numpy.org/>. Acesso em: 21 out. 2023.

PAPANDRÉA, P. J.; PEREIRA, A. de S.; PAIVA, A. P. de. Os principais setores de emprego na Mesorregião do Sul/Sudoeste de Minas: uma análise

multivariada. **Produção Online**, Florianópolis, SC, v. 22, n. 4, p. 3528-3554, 2022. Disponível em: <https://www.producaoonline.org.br/rpo/article/view/4716/2242>. Acesso em: 21 out. 2023.

TIPOS embutidos. **Python 3.12.2 Documentation**, [s. d.]. Disponível em: <https://docs.python.org/pt-br/3/library/stdtypes.html>. Acesso em: 21 out. 2023.

Aula 3

Classes e Métodos em Python

Classes e métodos em Python

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Quando pensamos em programação, devemos nos atentar à orientação a objetos, um paradigma essencial nesse contexto, pois organiza o código em torno de objetos, cada um representando entidades do mundo real.

Nesta etapa de aprendizagem, descobriremos, em Python, como criar classes, que são os modelos para a construção de objetos, e de que maneira é possível definir atributos e métodos dentro delas.

Discutiremos, também, sobre a herança em Python, a qual permite que classes-filhas herdem atributos e métodos de classes-pai, promovendo a reutilização de código e a criação de hierarquias de classes.

Você obterá uma compreensão sólida sobre esses conceitos fundamentais relativos à orientação a objetos e estará pronto para criar estruturas de código mais organizadas e reutilizáveis em Python.

Imagine, agora, a seguinte situação: você precisa criar um algoritmo que, a partir de características imputadas, mostre um resumo e o status de um veículo. Você também deve fazer algo semelhante para promover uma subdivisão dessa classe veículos. Vamos, juntos, resolver essa demanda?

Vamos Começar!

Introdução à orientação a objetos

Na indústria de software, a evolução é constante, com tecnologias que mudam rapidamente, mas os conceitos fundamentais do paradigma de programação orientada a objetos permanecem sólidos e são cruciais. Compreender esses conceitos é essencial, pois permite que você os implemente em qualquer tecnologia adotada pela sua empresa, independentemente de mudanças frequentes.

A programação orientada a objetos tem suas raízes fincadas na década de 1960, mas só ganhou destaque a partir de 1990. Uma linguagem de programação é considerada orientada a objetos quando incorpora os princípios de abstração e suporta o uso de encapsulamento, herança e polimorfismo.

No entanto, para entender completamente a programação orientada a objetos, é importante compreender o que são objetos e qual é o papel das classes.

Os programas são construídos em torno de objetos, que são as unidades fundamentais. Uma classe atua como um modelo para um objeto. Pode-se pensar em uma classe como o projeto de uma casa, no qual um arquiteto define todos os detalhes da estrutura. A classe organiza os dados e comportamentos que os objetos de uma classe específica terão.

Confira, a seguir, um exemplo de classe:

Classe: Pessoa

Atributos (dados):

Nome:

Idade:

Gênero:

Métodos (comportamentos):

Cumprimentar: saúda como “Olá, meu nome é”.

Aniversário: aumenta a idade em 1.

Objeto 1: Pessoa1

Atributos (dados):

Nome: João

Idade: 30

Gênero: Masculino

Métodos (comportamentos):

Cumprimentar: saúda como “Olá, meu nome é João”.

Aniversário: aumenta a idade em 1.

Nesse exemplo, a classe “Pessoa” define os atributos (nome, idade, gênero) e métodos (cumprimentar, aniversário) que o objeto “Pessoa1” pode usar.

Cada objeto tem seus próprios atributos, mas compartilha os mesmos métodos da classe. Isso demonstra como a programação orientada a objetos modela entidades do mundo real.

Para as classes, temos os seguintes “componentes” principais:

- **Atributos:** são os dados que representam o estado do objeto, como nome e idade.
- **Métodos:** definem o comportamento do objeto, indicando as ações que ele pode executar, como cumprimentar ou fazer login.
- **Encapsulamento:** combina atributos e métodos em uma entidade, permitindo controlar o acesso a atributos por meio de métodos.
- **Herança:** possibilita que uma classe herde atributos e métodos de outra, promovendo o reúso de código e a organização hierárquica, como na relação entre as classes pessoa, funcionário e cliente.
- **Polimorfismo:** refere-se à capacidade de várias classes responderem de forma diferente a uma mesma mensagem, graças à herança e às respostas específicas de cada classe às mensagens.

Classes em Python

Python é uma linguagem que oferece suporte ao paradigma orientado a objetos, viabilizando a implementação de encapsulamento, herança e polimorfismo. A criação de uma classe em Python é feita com a palavra reservada “class”, seguida do nome da classe, e em um bloco indentado são definidos os atributos e métodos.

```
# Define uma classe chamada Pessoa.
class Pessoa:
    # O método __init__ é um construtor, chamado quando um objeto da classe é criado.
    # Ele inicializa os atributos da classe.
    def __init__(self, nome, idade, genero):
        # self é uma convenção em Python que se refere à própria instância da classe.
        # Os parâmetros nome, idade e gênero são passados durante a criação do objeto.
        # Eles são usados para inicializar os atributos da instância.
        self.nome = nome # Atribui o valor de nome ao atributo nome da instância.
        self.idade = idade # Atribui o valor de idade ao atributo idade da instância.
        self.genero = genero # Atribui o valor de gênero ao atributo gênero da instância.
    # O método cumprimentar retorna uma saudação com o nome da pessoa.
    def cumprimentar(self):
        return f"Olá, meu nome é {self.nome}."
    # O método aniversario aumenta a idade da pessoa em 1.
    def aniversario(self):
```

```
self.idade += 1
# Cria uma instância da classe "Pessoa" com os valores "João", 30 e "Masculino" para nome,
idade e gênero, respectivamente.
pessoa1 = Pessoa("João", 30, "Masculino")
# Chama o método "cumprimentar" na instância pessoa1 e imprime a saudação.
print(pessoa1.cumprimentar()) # Saída: "Olá, meu nome é João."
# Acessa o atributo idade da instância pessoa1 e imprime sua idade.
print(f"Idade: {pessoa1.idade}") # Saída: "Idade: 30"
# Chama o método "aniversário" na instância pessoa1 para aumentar sua idade em 1.
pessoa1.aniversario()
# Acessa o atributo idade atualizado da instância pessoa1 e imprime a nova idade.
print(f"Nova idade: {pessoa1.idade}") # Saída: "Nova idade: 31"
```

Nesse exemplo, criamos a classe "Pessoa" com os atributos nome, idade e gênero, bem como os métodos cumprimentar e aniversario. Depois, construímos uma instância da classe "pessoa1" e demonstramos como acessar os atributos e chamar os métodos dessa instância.

O construtor da classe `__init__()` é capaz de receber um valor diferente para cada objeto, o que é de suma importância na construção da classe. Note que nesse caso determinamos dois tipos diferentes de atributos: duas strings (nome e gênero) e um int (idade).

Herança em Python

A herança é um dos pilares fundamentais da programação orientada a objetos, pois permite que uma classe (a classe-filha) herde características e comportamentos de outra classe (a classe-pai). Em Python, essa técnica é amplamente suportada e flexível, possibilitando que uma classe-filha herde de múltiplas classes-pai, processo que configura um conceito conhecido como herança múltipla.

A sintaxe para criar uma classe-filha que herda de uma classe-pai é simples e legível. A classe-filha é definida após o nome da classe-pai, entre parênteses.

Acompanhe, a seguir, a forma básica:

```
class ClasseFilha(ClassePai):
# Definição da classe-filha

class ClasseFilha(ClassePai1, ClassePai2, ClassePai3):
# Definição da classe-filha
```

Siga em Frente...

Benefícios da herança

1. **Reutilização de código:** a herança permite que você reutilize o código existente, aproveitando a estrutura e a funcionalidade de classes-pai em suas subclasses.
2. **Extensibilidade:** você pode estender ou adicionar comportamentos específicos às classes-filhas sem modificar as classes-pai, mantendo a coesão e a organização do código.
3. **Hierarquia de classes:** é possível criar uma hierarquia de classes na qual classes-filhas podem herdar características comuns de classes-pai e, por sua vez, serem herdadas por outras classes.

Imagine um cenário no qual tenhamos uma classe-pai chamada “Animal” com atributos e métodos gerais para representar qualquer animal. Podemos criar classes-filhas, como “Cachorro” e “Gato,” que herdam essas características gerais, mas que também podem ter comportamentos específicos, como latir e miar, respectivamente. Dessa forma, aproveitamos a reutilização de código e estendemos funcionalidades de acordo com a necessidade.

```
class Animal:
    def __init__(self, nome):
        self.nome = nome
    def fazer_barulho(self):
        pass
class Cachorro(Animal):
    def fazer_barulho(self):
        return "Latir"
class Gato(Animal):
    def fazer_barulho(self):
        return "Miar"

# Criando objetos das classes-filhas
rex = Cachorro("Rex")
whiskers = Gato("Whiskers")
# Chamando o método fazer_barulho em objetos
print(f"{rex.nome} faz: {rex.fazer_barulho()}") # Saída: "Rex faz: Latir"
print(f"{whiskers.nome} faz: {whiskers.fazer_barulho()}") # Saída: "Whiskers faz: Miar"
```

Nesse exemplo, criamos objetos “rex” e “whiskers” das classes-filhas “Cachorro” e “Gato”, respectivamente. Em seguida, chamamos o método fazer_barulho() em cada objeto para determinar o som que cada animal faz. Isso ilustra a herança em ação, quando as classes-filhas herdam o método da classe-pai, mas podem fornecer suas próprias implementações.

Vamos Exercitar?

Agora é a hora de criar nossa classe e fazer a aplicação dos conhecimentos obtidos nesta aula. Vamos lá! Primeiro, devemos trazer as características (atributos) da classe veículo: marca, modelo e ano. Para os métodos, vamos adicionar a velocidade a partir da aceleração e diminuir a velocidade por meio da frenagem. Por fim, mostraremos os atributos e a velocidade atual do veículo.

```
class Veiculo:
    def __init__(self, marca, modelo, ano):
        self.marca = marca
        self.modelo = modelo
        self.ano = ano
        self.velocidade = 0

    def acelerar(self, incremento):
        self.velocidade += incremento

    def frear(self, decremento):
        self.velocidade -= decremento

    def status(self):
        return f"Marca: {self.marca}, Modelo: {self.modelo}, Ano: {self.ano}, Velocidade: {self.velocidade} km/h"

class Carro(Veiculo):
    def __init__(self, marca, modelo, ano, potencia):
        super().__init__(marca, modelo, ano)
        self.potencia = potencia

    def acelerar(self, incremento):
        # Carros podem acelerar mais rápido.
        self.velocidade += incremento + self.potencia

class Bicicleta(Veiculo):
    def __init__(self, marca, modelo, ano, tipo):
        super().__init__(marca, modelo, ano)
        self.tipo = tipo

    def status(self):
        return f"Marca: {self.marca}, Modelo: {self.modelo}, Ano: {self.ano}, Tipo: {self.tipo}, Velocidade: {self.velocidade} km/h"

# Criando objetos
carro1 = Carro("Toyota", "Corolla", 2022, 150)
bicicleta1 = Bicicleta("Trek", "Mountain Bike", 2021, "MTB")
```

```
# Acelerando e verificando o status
carro1.acelerar(50)
bicicleta1.acelerar(20)

# Exibindo o status dos veículos
print("Status do Carro:")
print(carro1.status())

print("\nStatus da Bicicleta:")
print(bicicleta1.status())
```

Temos a classe-pai “Veiculo”, com atributos e métodos gerais; a classe-filha “Carro”, que herda de “Veiculo” e inclui a potência no método de aceleração; e a classe-filha “Bicicleta”, que herda de “Veiculo” e inclui o tipo no método de status. Criamos objetos de carros e bicicletas, e demonstramos como eles podem herdar atributos e métodos da classe-pai, enquanto as classes-filhas podem fornecer suas próprias implementações.

Gostou dessa solução? Espero que sim! Rode esse código no [colab](#), faça modificações e “brinque” com ele. Lembre-se de que a prática é extremamente importante para alcançar melhorias.

Saiba mais

1. O livro [Introdução à computação usando Python: um foco no desenvolvimento de aplicações](#) apresenta uma introdução à programação, ao desenvolvimento de aplicações de computador e à ciência da computação. Logo, para você, que está iniciando seu aprendizado em Python, essa obra representa uma leitura importante.

PERKOVIC, L. **Introdução à computação usando Python**: um foco no desenvolvimento de aplicações. Rio de Janeiro: LTC, 2016.

2. Outra leitura interessante para quem está começando a programar em Python é a do livro [Começando a programar em Python para leigos](#).

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book.

3. Por fim, outra dica para estudo e aprofundamento sobre esse tema é o livro [Use a cabeça! Python](#).

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book.

Referências

9. CLASSES. **Python 3.12.2 Documentation**, 8 fev. 2020. Disponível em: <https://docs.python.org/pt-br/3/tutorial/classes.html>. Acesso em: 25 out. 2023.

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em <https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

GOOGLE COLAB. **Página inicial**, [s. d.]. Disponível em: <https://colab.research.google.com/>. Acesso em: 25 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos**: lógica para desenvolvimento de programação de computadores. 29. ed. São Paulo: Érica, 2019.

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555202298>. Acesso em: 12 out. 2023.

PERKOVIC, L. **Introdução à computação usando Python**: um foco no desenvolvimento de aplicações. Rio de Janeiro: LTC, 2016.

Aula 4

Bibliotecas e Módulos em Python

Bibliotecas e módulos em Python

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Partida

Que a linguagem Python tem suas qualidades você já sabe, agora vamos começar a investigar conceitos e ferramentas que fazem da Python uma potência atual quando se trata de programação.

Você estudará sobre módulos e biblioteca em Python, que são componentes de código que servem como conjuntos de funções em Python, os quais facilitam a organização do código e a reutilização de funções em várias aplicações.

Esses módulos são classificados em três tipos: built-in, de terceiros e próprios. O primeiro módulo é "pronto" e já vem na instalação do Python. Os módulos de terceiros são produzidos por desenvolvedores e disponibilizados via PyPI. Por fim, os próprios consistem na construção de nós para resolver um determinado problema e podem ser reutilizados.

Também vamos examinar um módulo/biblioteca de terceiros: o Matplotlib, que é uma biblioteca de visualização, uma das mais populares em Python, vale ressaltar.

Suponha que você precise visualizar a contagem de venda de um produto hipotético. Vamos usar os conhecimentos obtidos nesta aula para construir essa visualização?

Vamos Começar!

Módulos e biblioteca em Python

Existem duas abordagens principais para se organizar o código em Python: usando funções ou classes para encapsular funcionalidades; e dividindo o código em vários arquivos .py para modularizar a solução. O ideal é combinar essas técnicas, criando módulos separados em arquivos independentes. De acordo com a documentação oficial do Python, é recomendável separar funcionalidades que podem ser reutilizadas em módulos distintos. Essa abordagem de modularização ajuda a manter o código mais organizado e legível, facilitando a manutenção e a reutilização de componentes.

Mas, afinal, o que são módulos? São componentes de código que servem como bibliotecas ou conjuntos de funções em Python. Eles abrigam uma variedade de funcionalidades, incluindo operações matemáticas, interações com o sistema operacional e muitas outras atividades. Módulos representam uma maneira elegante e eficaz de reutilizar código em diferentes partes de um programa ou em projetos distintos.

Em Python, frequentemente ouvimos falar tanto de módulos quanto de bibliotecas. A relação entre esses elementos tem a ver com o fato de que, na prática, um módulo pode ser considerado uma biblioteca de códigos. Para ilustrar essa ideia, considere o módulo “math”, que oferece várias funções matemáticas, e o módulo “os”, que disponibiliza funções relacionadas ao sistema operacional, como obtenção do diretório de trabalho atual (getcwd), listagem de arquivos em um diretório (listdir), criação de pastas (mkdir), entre muitos outros recursos. Esses módulos são essencialmente bibliotecas de funções relacionadas a áreas específicas, como matemática e operações do sistema, viabilizando a reutilização eficiente e elegante de um código.

Como utilizar um módulo?

```
#primeiro modo  
import math
```

```
math.sqrt(25)  
math.log2(1024)  
math.cos(45)
```

```
#segundo modo  
import math as m
```

```
m.sqrt(25)  
m.log2(1024)  
m.cos(45)
```

```
#terceiro modo  
from math import sqrt, log2, cos  
  
sqrt(25)
```

```
log2(1024)
cos(45)

#resultado para todos: 0.5253219888177297
```

No primeiro modo, usamos a importação que carrega todas as funções na memória, trouxemos toda a funcionalidade de “math” e colocamos `math.sqrt`, por exemplo, para chamar a função `sqrt`.

No segundo modo, utilizamos a importação que carrega todas as funções na memória, mas, nesse caso, demos um apelido para o módulo. Utilizamos `m.sqrt`, por exemplo, para chamar a função `sqrt`.

No terceiro modo, usamos a importação que carrega funções específicas na memória, utilizando-a diretamente – `sqrt()`, por exemplo.

Classificação dos módulos (built-in, de terceiros e próprios)

Podemos classificar os módulos (bibliotecas) em três categorias:

1. **Módulos built-in:** embutidos no interpretador.
2. **Módulos de terceiros:** criados por terceiros e disponibilizados via PyPI.
3. **Módulos próprios:** criados pelo desenvolvedor.

Os módulos built-in fazem parte do núcleo da linguagem e estão disponíveis diretamente no interpretador, sem a necessidade de instalação adicional. São carregados automaticamente quando você inicia o interpretador Python e fornecem funcionalidades básicas que são comuns a muitos programas. Alguns exemplos de módulos built-in em Python são: **math; os; sys; Random; datetime; re; collections**.

Esses são apenas alguns exemplos dos muitos módulos built-in disponíveis em Python. Eles são amplamente utilizados em muitos programas Python e concedem funcionalidades essenciais para várias tarefas.

Os módulos de terceiros em Python são extensões de funcionalidade que não fazem parte da biblioteca-padrão do Python, mas são criados e mantidos por desenvolvedores externos à comunidade oficial do Python. Eles são frequentemente distribuídos por meio do Python Package Index (PyPI) e podem ser instalados no ambiente Python para adicionar funcionalidades extras aos seus programas. Confira, a seguir, alguns pontos importantes sobre módulos de terceiros:

1. Ampliam a funcionalidade do Python em diversas áreas, como na manipulação de dados, gráficos, interfaces gráficas, integração com bancos de dados e aprendizado de máquina.

2. A instalação é feita usando o gerenciador de pacotes padrão: pip. Exemplo: **pip install requests**.
3. Gerenciar dependências é essencial à medida que projetos crescem. O uso de um arquivo requirements.txt facilita a instalação de todas as dependências em um único comando pip.
4. Ambientes virtuais isolam projetos Python para evitar conflitos entre diferentes versões de módulos de terceiros.
5. Conhecer as licenças dos módulos de terceiros é importante, pois eles podem variar de código aberto a proprietário, e a qualidade da manutenção pode mudar.
6. Módulos de terceiros geralmente possuem comunidades ativas de desenvolvedores e documentação rica, fornecendo suporte e recursos valiosos.

Como exemplos de módulos de terceiros, podemos citar: NumPy, pandas e Matplotlib.

Já os módulos próprios, também conhecidos como módulos personalizados ou módulos definidos pelo usuário, são módulos em Python que você cria para organizar e reutilizar seu próprio código. Eles são uma parte importante da prática de desenvolvimento em Python, pois permitem dividir seu código em unidades lógicas, tornando-o mais legível, manutenível e reutilizável.

Vale destacar que os módulos próprios constituem-se como uma ferramenta poderosa para organizar, reutilizar e compartilhar códigos em Python. Eles permitem que você crie bibliotecas personalizadas para atender às necessidades específicas dos seus projetos e promovem a modularização, que é uma prática recomendada no desenvolvimento de software.

Siga em Frente...

Matplotlib

O Matplotlib é uma das bibliotecas de visualização mais populares em Python, já que oferece uma ampla gama de recursos para criar gráficos e visualizações de dados de maneira flexível e personalizável. É frequentemente usado para construir gráficos estáticos, interativos e até mesmo animações.

Para utilizar o Matplotlib, você normalmente precisa importar o módulo pyplot, que fornece uma interface de alto nível para criar gráficos de modo conveniente. Observe, a seguir, um exemplo simples de como elaborar um gráfico de linha usando o Matplotlib:

```
import matplotlib.pyplot as plt
```

```
# Dados
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 1, 3, 5]
```

```
# Criar um gráfico de linha
```

```
plt.plot(x, y)
```

```
# Adicionar rótulos aos eixos
```

```
plt.xlabel('Eixo X')
```

```
plt.ylabel('Eixo Y')
```

```
# Adicionar um título ao gráfico
```

```
plt.title('Exemplo de Gráfico de Linha')
```

```
# Mostrar o gráfico
```

```
plt.show()
```

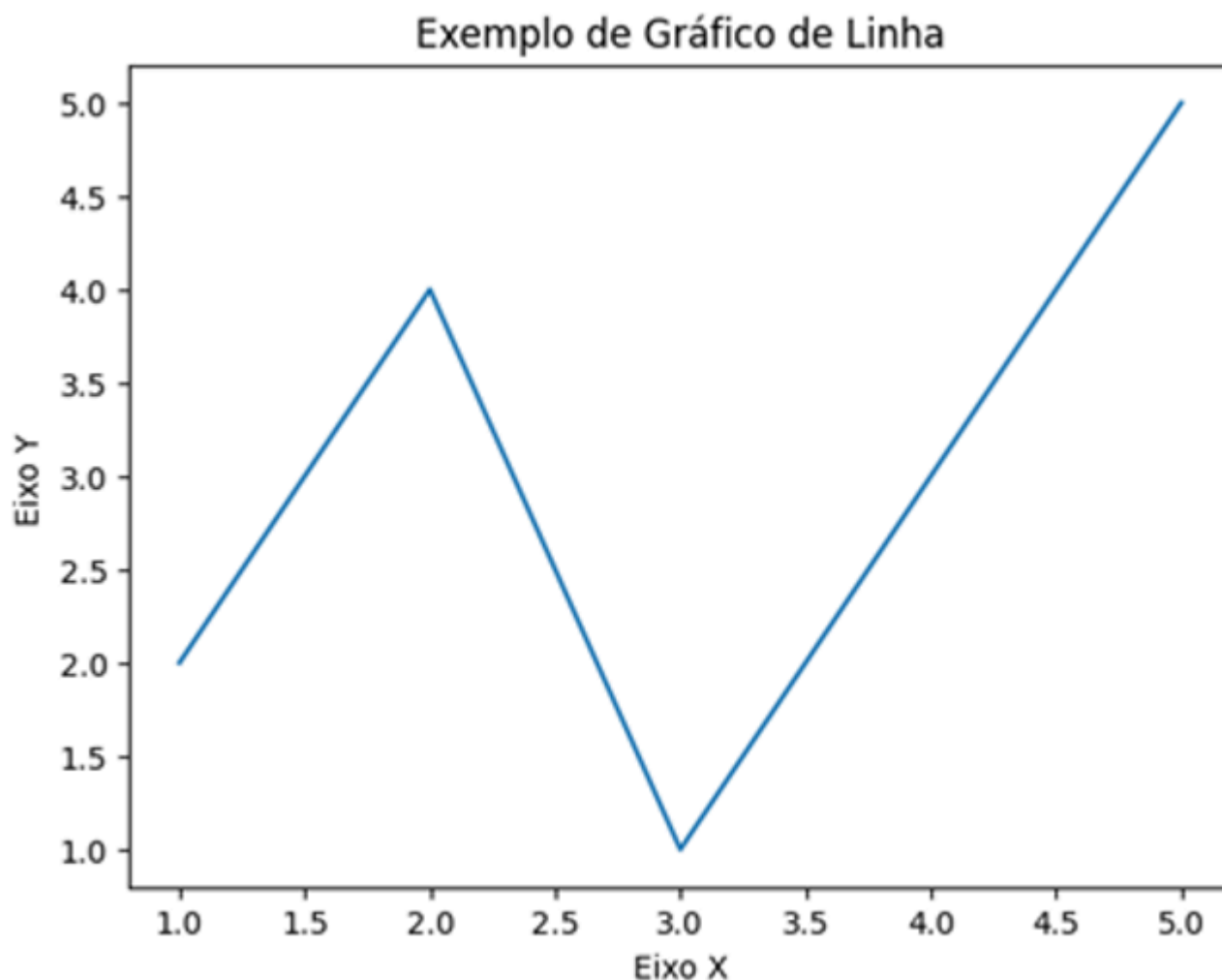


Figura 1 | Exemplo de gráfico de linha. Fonte: elaborada pelo autor.

Nesse exemplo:

1. Importamos o módulo **pyplot** do Matplotlib como **plt**.
2. Definimos listas **x** e **y**, que representam os pontos do gráfico.
3. Usamos **plt.plot(x, y)** para criar o gráfico de linha.
4. Adicionamos rótulos aos eixos X e Y com **plt.xlabel()** e **plt.ylabel()**.
5. Adicionamos um título ao gráfico com **plt.title()**.
6. Por fim, usamos **plt.show()** para exibir o gráfico.

O código apresentado anteriormente cria um simples gráfico de linha com os pontos (1, 2), (2, 4), (3, 1), (4, 3) e (5, 5), rotulando os eixos e dando um título ao gráfico.

O Matplotlib disponibiliza uma variedade de opções de personalização para gráficos, permitindo que você ajuste cores, estilos de linha, marcadores e muitos outros aspectos. Trata-se de uma biblioteca eficiente para criar gráficos de alta qualidade em Python, sendo amplamente utilizada na análise e visualização de dados.

Vamos Exercitar?

Para resolver o problema inicial, criaremos uma situação hipotética:

```
import matplotlib.pyplot as plt

# Dados de exemplo
meses = ['Janeiro', 'Fevereiro', 'Março', 'Abril', 'Maio']
vendas = [120, 90, 150, 80, 200]

# Criar um gráfico de barras
plt.bar(meses, vendas, color='royalblue')

# Adicionar rótulos aos eixos
plt.xlabel('Mês')
plt.ylabel('Vendas (em unidades)')

# Adicionar um título ao gráfico
plt.title('Vendas Mensais')

# Mostrar o gráfico
plt.show()
```

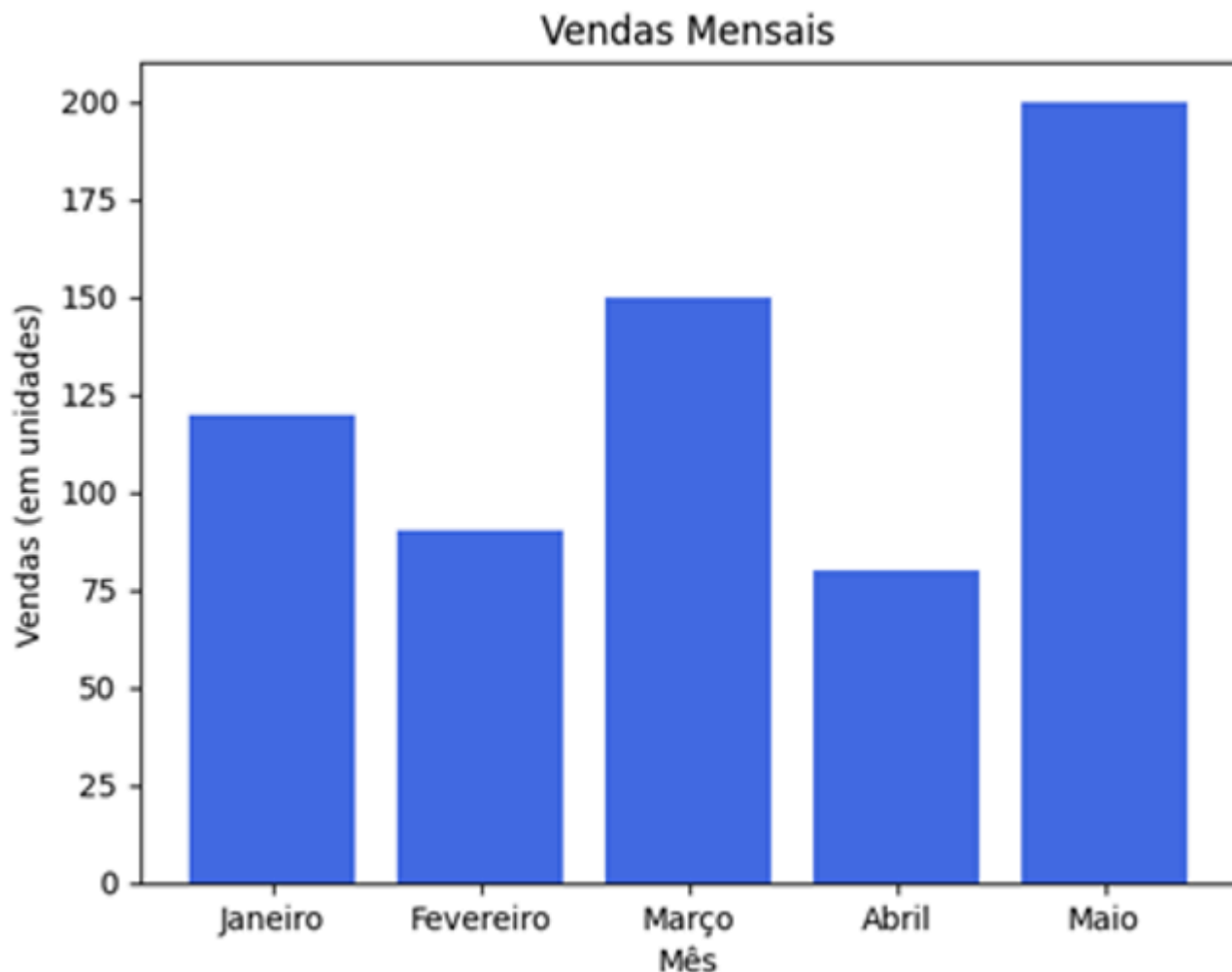


Figura 2 | Valores mensais. Fonte: elaborada pelo autor.

Esse código cria um gráfico de barras que exibe as vendas por mês. Você pode personalizar ainda mais esse gráfico ajustando cores, estilos e outros parâmetros de acordo com suas necessidades. O Matplotlib concede muitas opções de customização para criar gráficos visualmente atraentes.

Por meio desse conhecimento, você se sentirá cada vez mais preparado para elaborar soluções criativas e aplicá-las a diversas realidades. Lembre-se de sempre praticar!

Saiba mais

1. Para aprender mais detalhes sobre aplicações do Python, especialmente quanto ao Matplotlib, sugiro a leitura do artigo [Introdução à estilometria com Python](#). Nesse texto, apresentam-se análises estilométricas, que dizem respeito ao estudo quantitativo do estilo literário por meio de

métodos de leitura distante computacional. Para acessar o material sugerido, clique no link a seguir.

LARAMÉE, F. D. Introdução à estilometria com Python. **Programming Historian**, 21 abr. 2018.

2. Outra dica para estudo e aprofundamento sobre esse tema é o livro [Use a cabeça! Python](#).

BARRY, P. **Use a cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book.

3. Para entender como funciona a aplicação do Python em data science, sugiro a leitura do capítulo 3 do livro [Data science do zero](#).

GRUS, J. **Data science do zero: primeiras regras com o Python**. Rio de Janeiro: Alta Books, 2021. E-book.

Referências

6. MÓDULOS. **Python 3.12.2 Documentation**, [s. d.]. Disponível em: <https://docs.python.org/pt-br/3/tutorial/modules.html>. Acesso em: 25 out. 2023.

BARRY, P. **Use a cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

GRUS, J. **Data science do zero: primeiras regras com o Python**. Rio de Janeiro: Alta Books, 2021. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788550816463>. Acesso em: 12 out. 2023.

LARAMÉE, F. D. Introdução à estilometria com Python. **Programming Historian**, 21 abr. 2018. Disponível em <https://programminghistorian.org/pt/licoes/introducao-estilometria-python>. Acesso em: 12 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29. ed. São Paulo: Érica, 2019.

Aula 5

Explorando Recursos do Python

Videoaula de Encerramento

Este conteúdo é um vídeo!



Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Dica para você

Aproveite o acesso para baixar os slides do vídeo, isso pode deixar sua aprendizagem ainda mais completa.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?
Bons estudos!

Ponto de Chegada

Olá, estudante! Para desenvolver a competência associada a esta unidade de aprendizagem, que é “Identificar os pilares da orientação a objetos e sua utilização na linguagem de programação com Python”, devemos, antes de tudo, conhecer os conceitos relacionados à estrutura dos dados em Python. Para isso, é necessário saber que a linguagem Python é baseada na orientação a objetos. Sendo assim, conhecer os tipos de objetos e suas especificidades é um ponto crucial nesse contexto.

Ao longo desta etapa de estudos, foi possível conectar esses conceitos fundamentais com a construção de scripts. Você também aprendeu a utilizar diferentes tipos de objeto para resolver problemas em diversas situações. Além disso, conhecemos o conceito de classe e sua importância para um código bem estruturado, o qual possa ser reutilizado e modificado para contextos distintos (Manzano; Oliveira, 2019).

Um ponto que merece destaque nesse cenário são as bibliotecas em Python, que podem surgir como “ferramentas” prontas (built-in ou de terceiros) ou criadas de modo personalizado. Existe uma grande variedade de bibliotecas, então a prática e a pesquisa para saber qual delas utilizar são aspectos essenciais para aumentar a gama de “ferramentas” disponíveis do Python.

Durante este processo de aprendizagem, você não apenas assimilou os conceitos e técnicas apresentados, mas também os colocou em prática por meio de situações do mundo real (Grus, 2021). A construção de scripts utilizando recursos como as bibliotecas Python, além de

aprimorar seu conhecimento, desenvolve sua habilidade de decompor problemas complexos em etapas lógicas e de criar soluções algorítmicas para esses casos (Perkovic, 2016).

A competência desta unidade de aprendizagem permite que você se torne um solucionador de problemas com proficiência em Python, preparando-o para enfrentar desafios tecnológicos e computacionais de forma eficaz.

É Hora de Praticar!



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Para contextualizar sua aprendizagem, imagine a seguinte situação: você está desenvolvendo um programa simples para gerenciar informações sobre livros em uma biblioteca e fazer uma contagem de livros por ano de publicação.

Questões norteadoras:

1. Como você pode aplicar seus conhecimentos em programação em Python para gerenciar essas informações?
2. Como é possível criar classes e utilizar bibliotecas para automatizar esse gerenciamento?

Para encerrar e consolidar seu aprendizado, reflita sobre as seguintes perguntas:

1. Como as estruturas de dados em Python podem ser usadas para tomar decisões em programas de forma correta?
2. Qual é a importância de reutilizar e modificar classes para otimizar códigos em Python?
3. Como você pode aplicar o conhecimento das estruturas dos dados e das bibliotecas do Python para resolver problemas complexos em sua trajetória acadêmica e profissional?

Essas considerações ajudarão você a incorporar de maneira mais profunda o conhecimento adquirido e a compreender o alcance de suas aplicações. Desejo a você muito sucesso em sua jornada de aprendizagem!

Vamos resolver o desafio seguindo um passo a passo.

Nesse estudo de caso, usaremos estruturas de dados em Python, bibliotecas, orientação a objetos e classes para criar um sistema básico de catalogação de livros.

Confira, a seguir, o código Python para criar o sistema de catalogação:

```
import matplotlib.pyplot as plt
# Classe para representar um livro
class Livro:
    def __init__(self, titulo, autor, ano_publicacao):
        self.titulo = titulo
        self.autor = autor
        self.ano_publicacao = ano_publicacao

    def __str__(self):
        return f'{self.titulo} por {self.autor}, Publicado em {self.ano_publicacao}'

# Criar uma lista de livros
biblioteca = []
# Função para adicionar um livro à biblioteca
def adicionar_livro(titulo, autor, ano_publicacao):
    novo_livro = Livro(titulo, autor, ano_publicacao)
    biblioteca.append(novo_livro)
    print(f'{titulo}' foi adicionado à biblioteca.")
# Função para listar todos os livros na biblioteca
def listar_livros():
    print()
    for livro in biblioteca:
        print(livro)
# Adicionar alguns livros à biblioteca
adicionar_livro(, , 1605)
adicionar_livro(, , 1813)
adicionar_livro(, , 1949)
adicionar_livro(, , 1967)
adicionar_livro(, , 1951)

# Listar todos os livros na biblioteca
listar_livros()
# Criar um gráfico de livros por ano
anos = list(set(anos)) # Remove duplicatas dos anos
anos.sort()

# Contagem de livros por ano
contagem_por_ano = [anos.count(ano) for ano in anos]

# Criar um gráfico de linha
plt.plot(anos, contagem_por_ano, marker='o', linestyle='-')
plt.xlabel('Ano de Publicação')
plt.ylabel('Número de Livros')
plt.title('Distribuição de Livros na Biblioteca por Ano de Publicação')
```

```
# Adicionar rótulos aos pontos de dados
for i, valor in enumerate(contagem_por_ano):
    plt.text(anos[i], valor, str(valor), ha='center', va='bottom')

plt.grid(True)

plt.show()
```

O resultado é:

O livro 'Dom Quixote' foi adicionado à biblioteca.
O livro 'Orgulho e Preconceito' foi adicionado à biblioteca.
O livro '1984' foi adicionado à biblioteca.
O livro 'Cem Anos de Solidão' foi adicionado à biblioteca.
O livro 'Apanhador no Campo de Centeio' foi adicionado à biblioteca.
Livros na Biblioteca:
Dom Quixote por Miguel de Cervantes, Publicado em 1605
Orgulho e Preconceito por Jane Austen, Publicado em 1813
1984 por George Orwell, Publicado em 1949
Cem Anos de Solidão por Gabriel Garcia Marquez, Publicado em 1967
Apanhador no Campo de Centeio por J.D. Salinger, Publicado em 1951



Figura 1 | Distribuição de livros na biblioteca por ano de publicação. Fonte: elaborada pelo autor. Esse exemplo demonstra como você pode aplicar os conceitos de classes, orientação a objetos e estruturas de dados em Python para criar um sistema simples de gerenciamento de livros em uma biblioteca. A utilização de bibliotecas Python adicionaria certas funcionalidades, como salvar e carregar informações de livros a partir de arquivos, bem como interfaces gráficas para a interação do usuário.

O material visual a seguir esquematiza os principais tópicos abordados nesta unidade de aprendizagem, na qual tratamos dos recursos da linguagem Python. Este infográfico exibe uma percepção clara e sucinta de cada parte desta etapa de estudos, enfatizando os conceitos e fundamentos necessários para uma boa compreensão dos saberes desenvolvidos.



Figura 2 | Infográfico: explorando recursos do Python. Fonte: elaborada pelo autor.

GRUS, J. **Data science do zero**: primeiras regras com o Python. Rio de Janeiro: Alta Books, 2021. E-book.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos**: lógica para desenvolvimento de programação de computadores. 29. ed. São Paulo: Érica, 2019.

PERKOVIC, L. **Introdução à computação usando Python**: um foco no desenvolvimento de aplicações. Rio de Janeiro: LTC, 2016.